

MONITORIZACIÓN Y GESTIÓN DE USUARIOS Y CURSOS EN UNA PLATAFORMA WEB DE PROGRAMACIÓN DE ROBOTS

Autora: Iraide Hoyas Puente

Tutor: José María Cañas Plaza

Cotutor: José Felipe Ortega Soto

ÍNDICE



INTRODUCCIÓN



OBJETIVOS



TECNOLOGÍAS



GESTIÓN DE USUARIOS Y CONTENIDOS



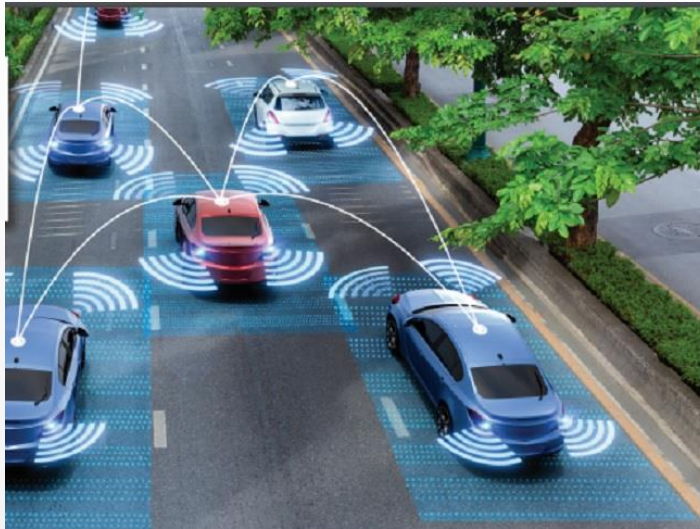
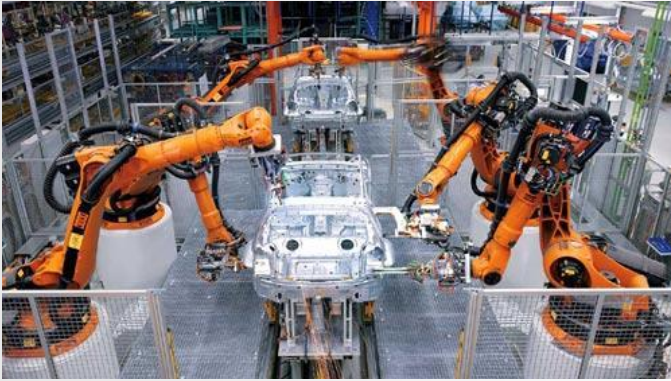
FUNCIONALIDADES PARA PROFESORES



CONCLUSIONES

INTRODUCCIÓN

- La robótica y sus aplicaciones



INTRODUCCIÓN

Middleware robóticos

- ROS

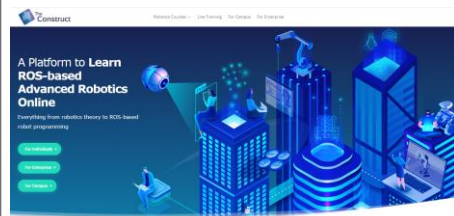
Simuladores

- Gazebo
- CoppeliaSim
- WeBots
- MatLab Robotics System Toolbox

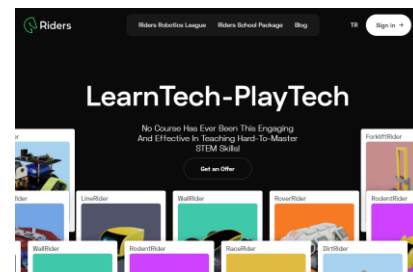
INTRODUCCIÓN

PLATAFORMAS DE EDUCACIÓN ROBÓTICA

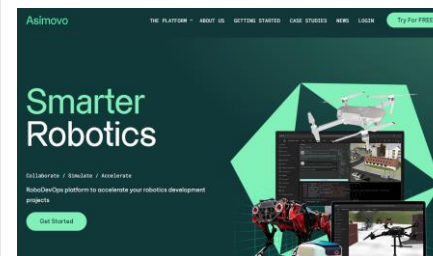
The Construct



Riders



Asimovo



Unibotics



INTRODUCCIÓN

TECNOLOGÍAS WEB

Conjunto de herramientas que permiten la comunicación entre computadoras mediante el uso de Internet.

Tecnologías del lado del servidor (backend)

Django

PHP

Bases de datos

Diferentes lenguajes de programación

Tecnologías del lado del cliente (frontend)

HTML

CSS

JavaScript

OBJETIVOS



MEJORAR LA GESTIÓN DE USUARIOS Y CONTENIDOS

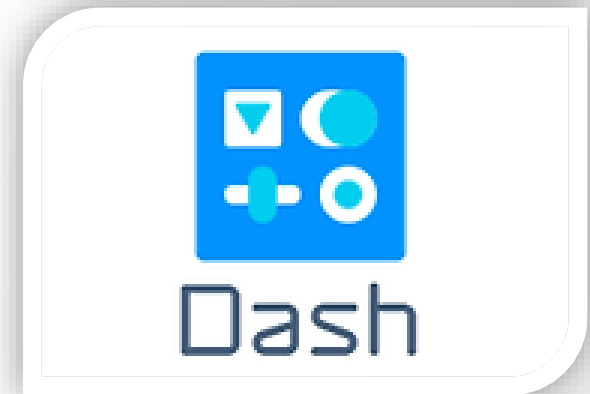
- Incluir roles
- Crear clases y cursos
- Incluir permisos
- Crear una nueva página de acceso



CREAR NUEVAS FUNCIONALIDADES PARA PROFESORES

- Monitorizar y visualizar la actividad de sus alumnos
- Visualizar y ejecutar el código de sus alumnos

TECNOLOGÍAS UTILIZADAS



TECNOLOGÍAS UTILIZADAS



TECNOLOGÍAS UTILIZADAS



Plataforma web orientada a la enseñanza de robótica

Programación de diferentes robots en Python

Uso de ROS

Uso de Gazebo

Acceso desde el propio navegador y para cualquier sistema operativo

Todas las dependencias ya están preinstaladas gracias a RADI

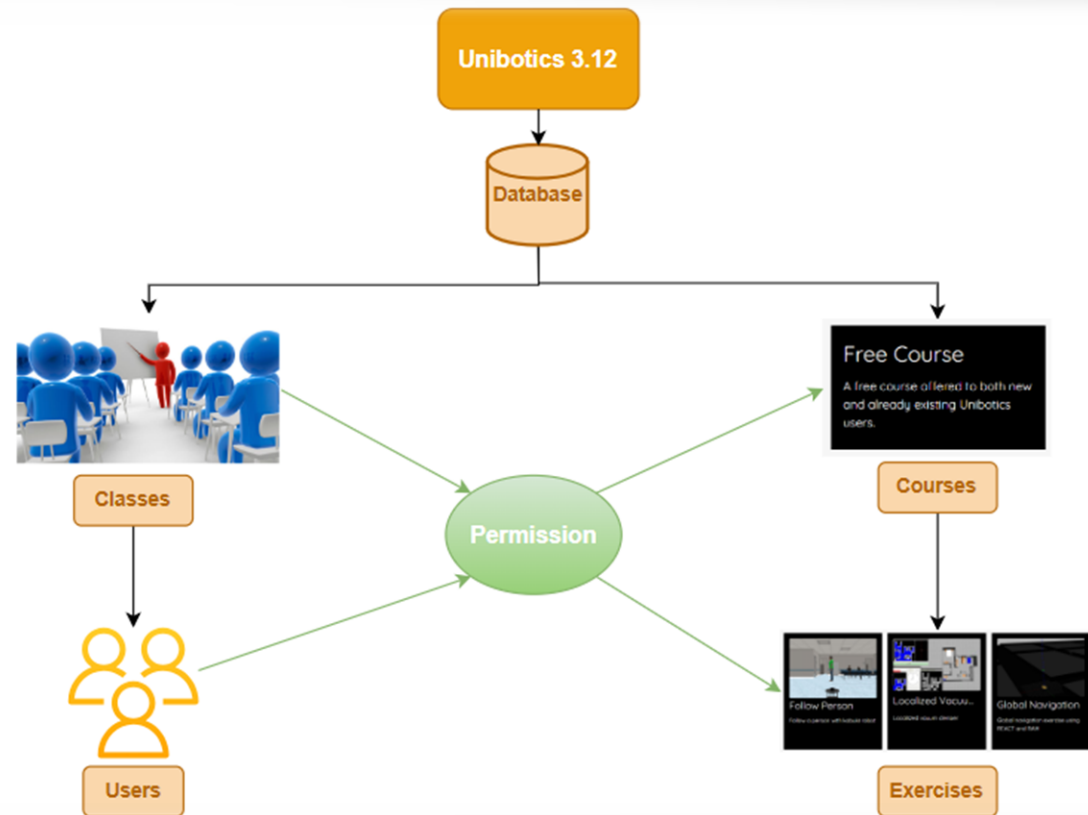
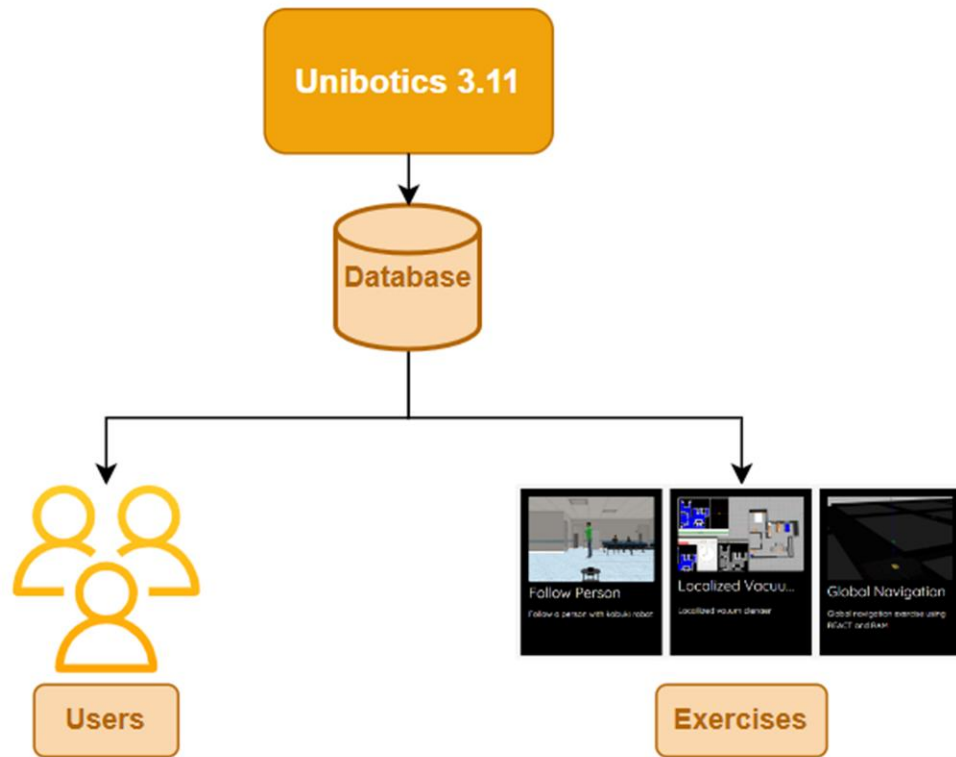
Django como servidor web

PostgreSQL como base de datos para información estructural

Amazon S3 para almacenar el código de los usuarios

GESTIÓN DE USUARIOS Y CONTENIDOS

- Diseño



GESTIÓN DE USUARIOS Y CONTENIDOS

- Modificaciones en la base de datos

Roles

Clases

Cursos

```
class Courses(models.Model):
```

```
    class Meta:
```

```
        db_table = 'curso'
```

```
    id_curso = models.AutoField(primary_key=True)
```

```
    nombre_curso = models.CharField(max_length=100)
```

```
    descripcion = models.TextField()
```

```
    def __str__(self):
```

```
        return self.nombre_curso
```

```
class OrderExerciseCourse(models.Model):
```

```
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
```

```
    exercise = models.ForeignKey(Exercise, on_delete=models.CASCADE)
```

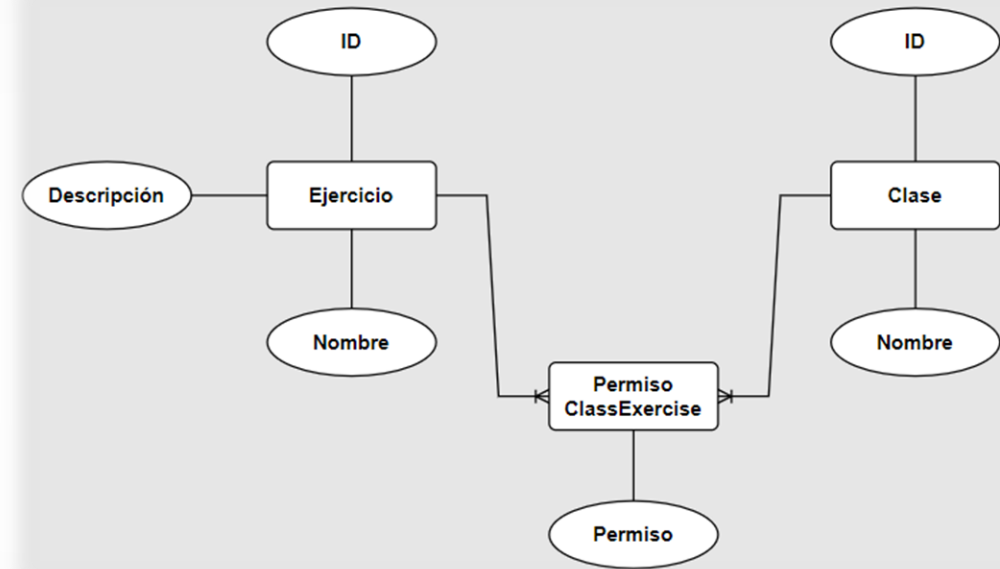
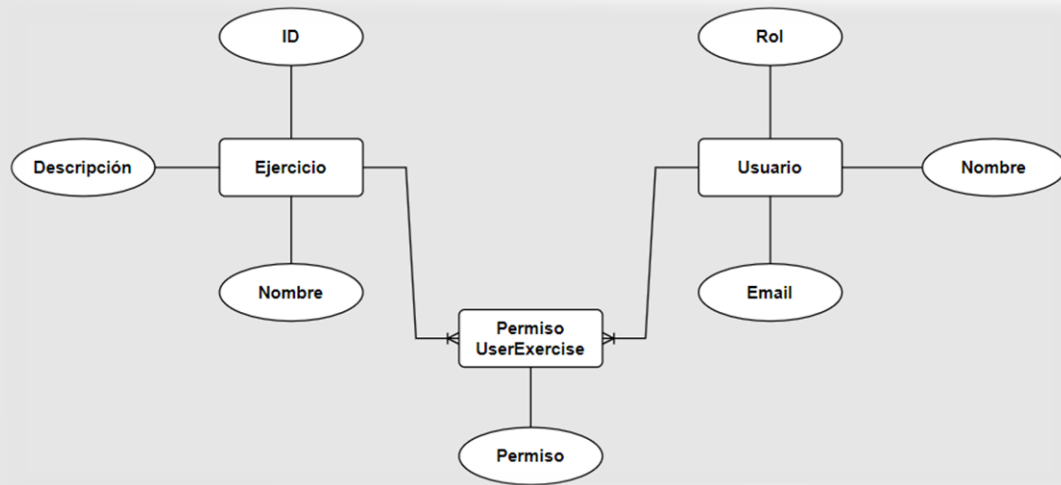
```
    order = models.PositiveIntegerField()
```

```
    class Meta:
```

```
        ordering = ['order']
```

GESTIÓN DE USUARIOS Y CONTENIDOS

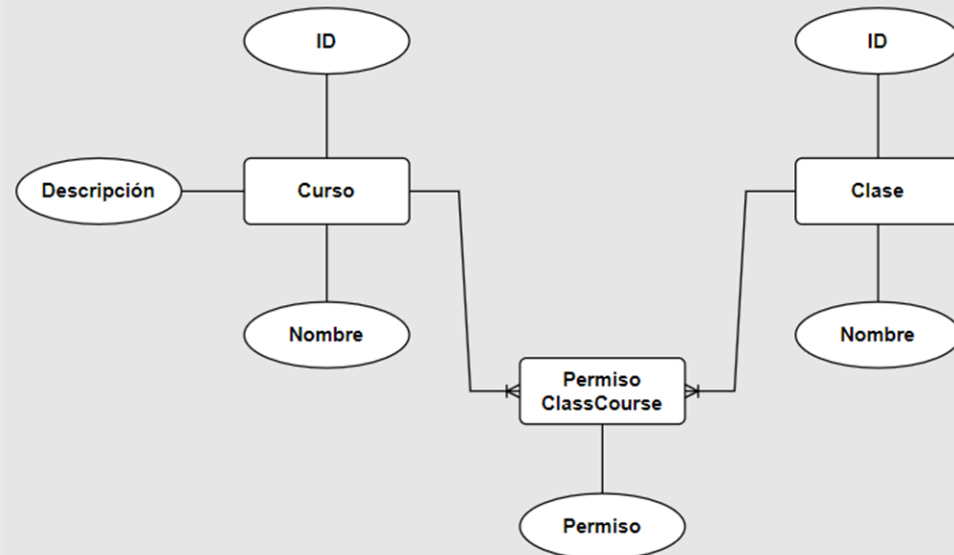
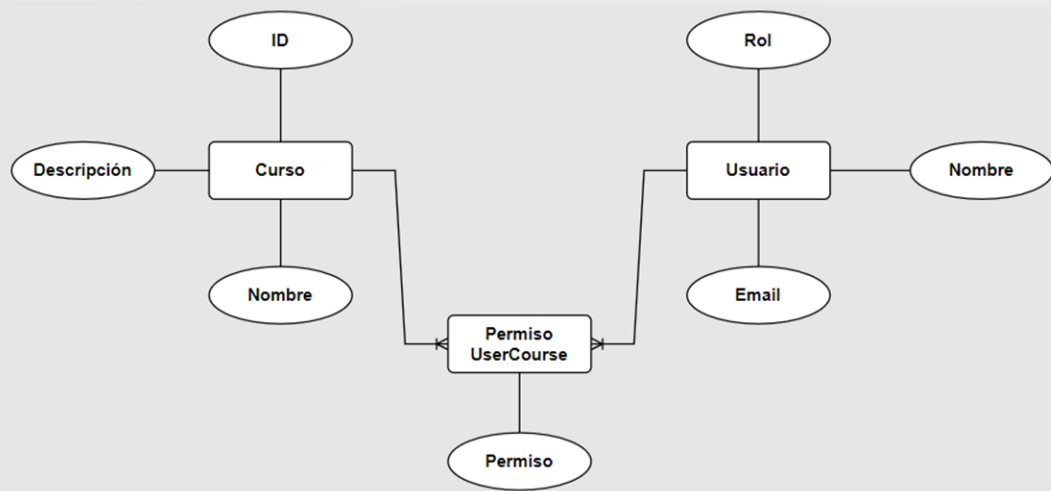
- Permisos de acceso a los ejercicios



```
class PermissionUserExercise(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE)
    exercise = models.ForeignKey(Exercise, on_delete=models.CASCADE)
    permission = models.BooleanField(default=False, help_text="True if user has permission to view the exercise. False otherwise")
```

GESTIÓN DE USUARIOS Y CONTENIDOS

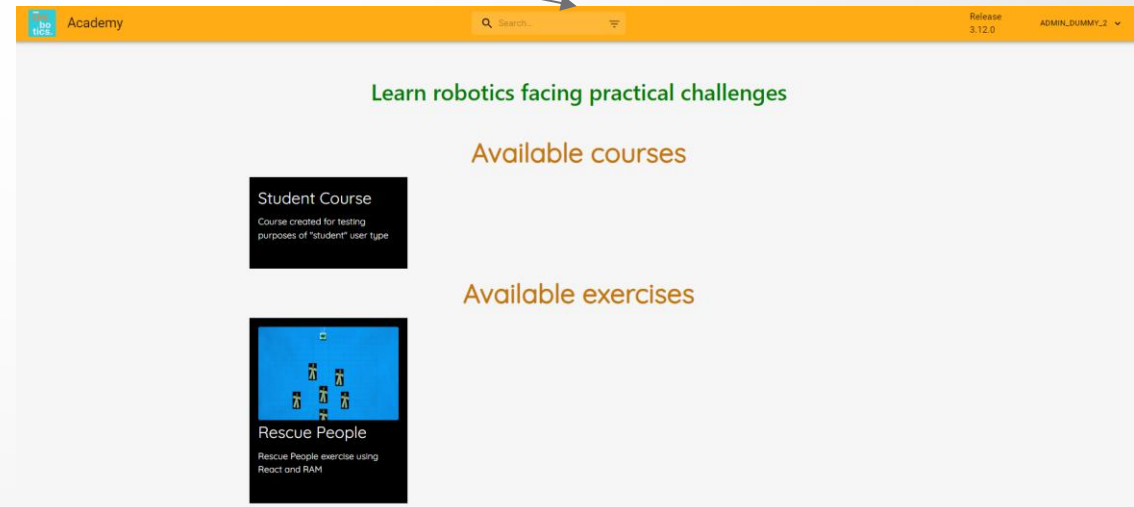
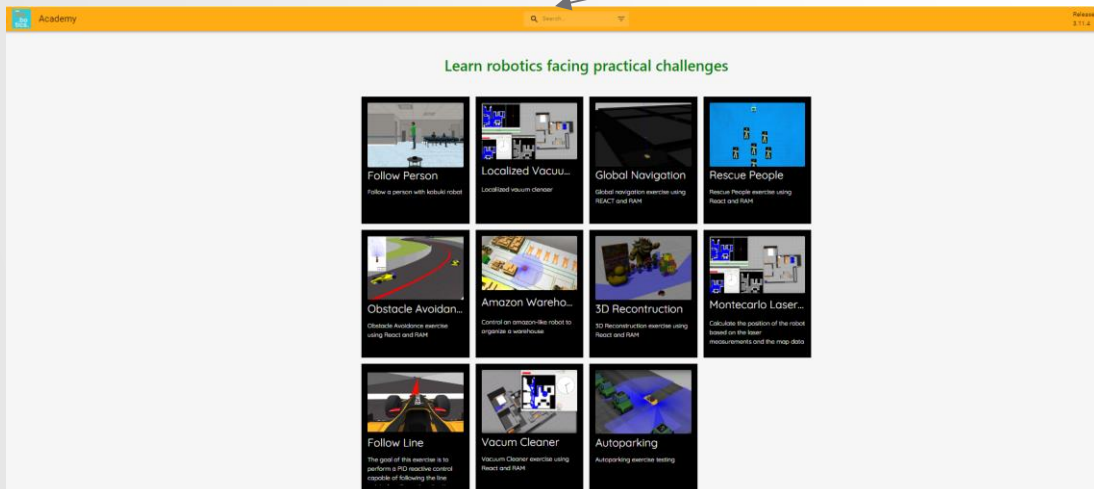
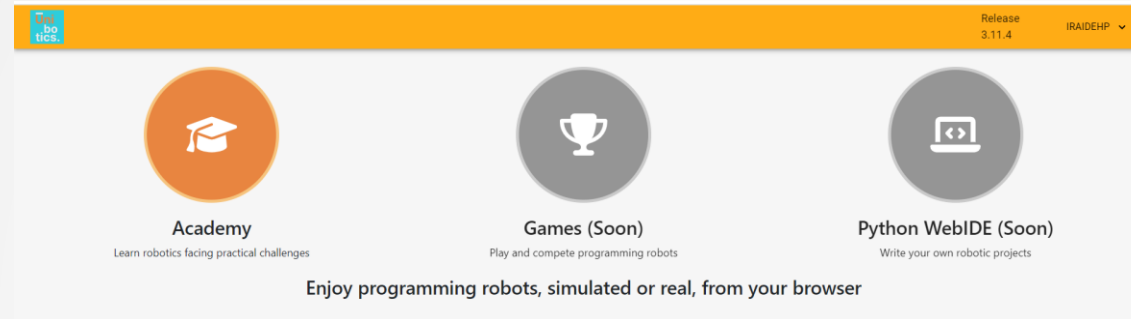
- Permisos de acceso a los cursos



```
class PermissionUserCourse(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE)
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
    permission = models.BooleanField(default=False, help_text="True if user has permission to view the course. False otherwise")
```

GESTIÓN DE USUARIOS Y CONTENIDOS

- Nueva interfaz de usuario



GESTIÓN DE USUARIOS Y CONTENIDOS

- ¿Cómo se muestran los contenidos al usuario en función de sus permisos?

```
def get_courses(request):
    user = request.user
    user_class = Classes.objects.filter(usuario=user)
    user_class = list(user_class.values('id_clase'))

    #obtener cursos a los que un usuario tiene permiso
    course_user = PermissionUserCourse.objects.filter(usuario=user, permission=True)
    course = list(course_user.values('course'))

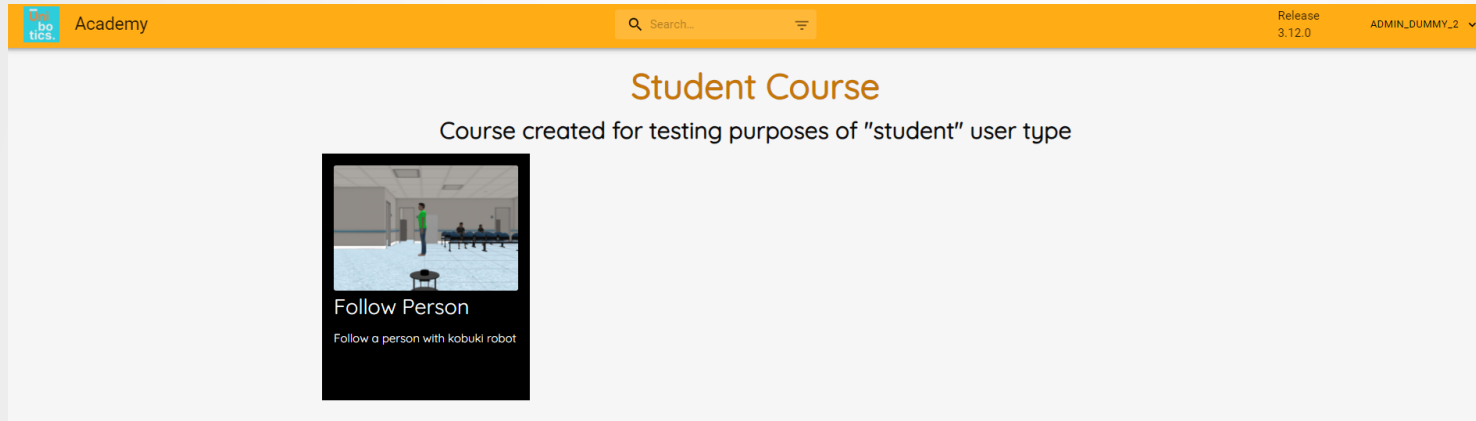
    #obtener cursos a los que un usuario tiene permiso por pertenecer a determinadas clases
    for c in user_class:
        course_class = PermissionClassCourse.objects.filter(clase=c['id_clase'], permission=True)
        course_class = list(course_class.values('course'))
        for cur in course_class:
            if cur not in course:
                course.append(cur)

    #añadir todos los cursos en el formato correcto para enviar los cursos disponibles
    courses_list = []
    for c in course:
        courses = Courses.objects.filter(id_curso=c['course'])
        courses = list(courses.values())
        courses_list.append(courses[0])

    return JsonResponse({"courses": courses_list})
```

GESTIÓN DE USUARIOS Y CONTENIDOS

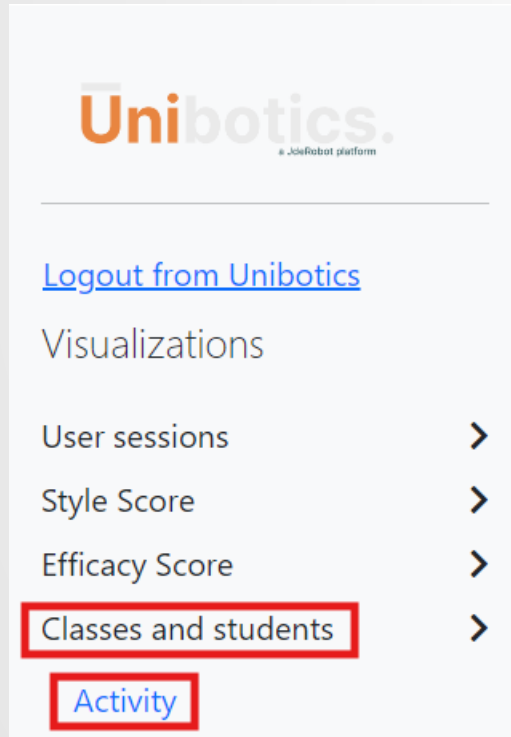
- Nueva página para los cursos



```
def course_information(request, courseid):
    course = Courses.objects.get(id_curso=courseid)
    name = course.nombre_curso
    description = course.descripcion
    return JsonResponse({"name": name, "description": description})
def get_exercise_list(request, courseid):
    course = Courses.objects.get(id_curso=courseid)
    order = OrderExerciseCourse.objects.filter(course=course)
    exercises_list = []
    for i in order:
        exercise = Exercise.objects.filter(exercise_id=i.exercise.exercise_id).values('exercise_id', 'name', 'description', 'tags', 'status')
        exercise = list(exercise)
        exercises_list.append(exercise[0])
    return JsonResponse(exercises_list, safe=False)
```

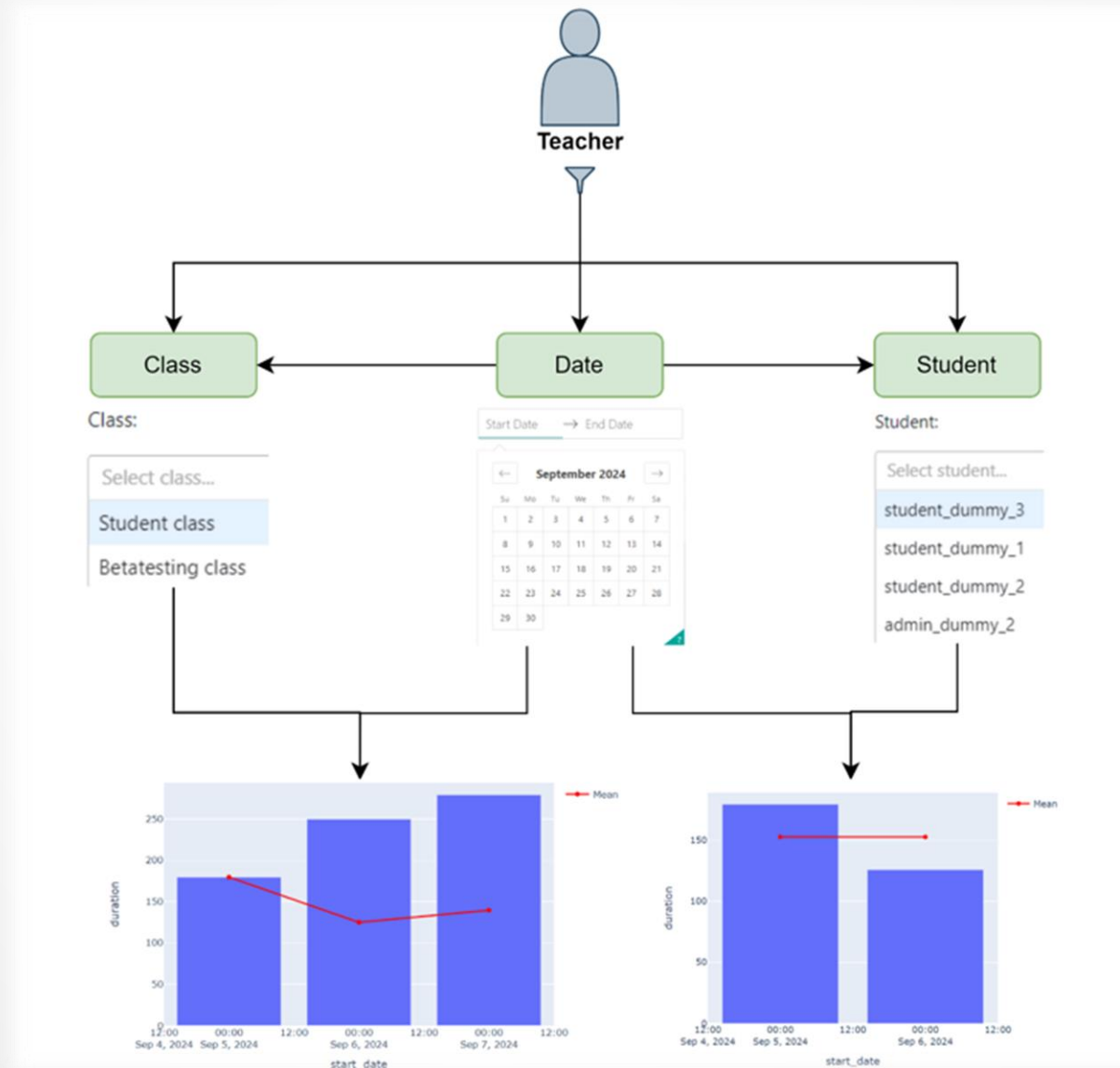
FUNCIONALIDADES PARA PROFESORES

- Monitorización y visualización de la actividad de los alumnos con DASH



FUNCIONALIDADES PARA PROFESORES

- Flujo que sigue el profesor para visualizar la actividad de sus alumnos



FUNCIONALIDADES PARA PROFESORES

Uso de tres
funciones para
obtener los datos
de actividad

```
def get_classes_and_students_name(conn, id):
```

```
def activity(conn, start_date, end_date, students, s, graph):
```

```
def get_temporal_figure_session_time(df):
```

```
    fig = px.bar(df, x='start_date', y='duration')
```

```
    fig.add_trace(go.Scatter(x=df['start_date'], y=df['mean'], line=dict(color='red'), name='Mean'),  
                    row=1, col=1)
```

```
    return fig
```

Actualización de
los datos al filtrar

```
@app.callback(
```

```
    dash.dependencies.Output('activity_class', 'figure'),
```

```
    [dash.dependencies.Input('actividad_clases', 'start_date'),
```

```
     dash.dependencies.Input('actividad_clases', 'end_date'),
```

```
     dash.dependencies.Input('classes', 'value')])
```

```
def update_output(start_date, end_date, clase):
```

FUNCIONALIDADES PARA PROFESORES

- Visualizar y ejecutar el código realizado por los alumnos

The screenshot displays the UniBotics Academy web interface. At the top, an orange header bar contains the 'UniBotics Academy' logo, status buttons for 'Robotics Backend REMOTE 4.6.6', 'World Simple', 'Visualization State READY', and 'Application FOLLOW LINE'. It also shows 'Release 3.12.0' and a user dropdown 'ADMIN_DUMMY_2'. Below the header, a red box highlights two dropdown menus: 'Select class...' and 'Select student...'. The main area is divided into three sections: a code editor on the left with a dark theme and pre-written Python code, a terminal at the bottom left showing a shell prompt, and a simulation window on the right. The simulation window shows a top-down view of a green track on a black background and a first-person view of a yellow and black robot car on a grey track. A progress bar at the top of the simulation window indicates '0%' completion. A toolbar with various icons and a 'Press Escape to exit Follow mode' message is visible above the first-person view.

UniBotics Academy

Robotics Backend
REMOTE 4.6.6

World
Simple

Visualization State
READY

Application
FOLLOW LINE

Release
3.12.0

ADMIN_DUMMY_2

Select class... Select student...

SAVE

1 import GUI
2 import HAL
3 # Enter sequential code!
4
5 while True:
6 # Enter iterative code!

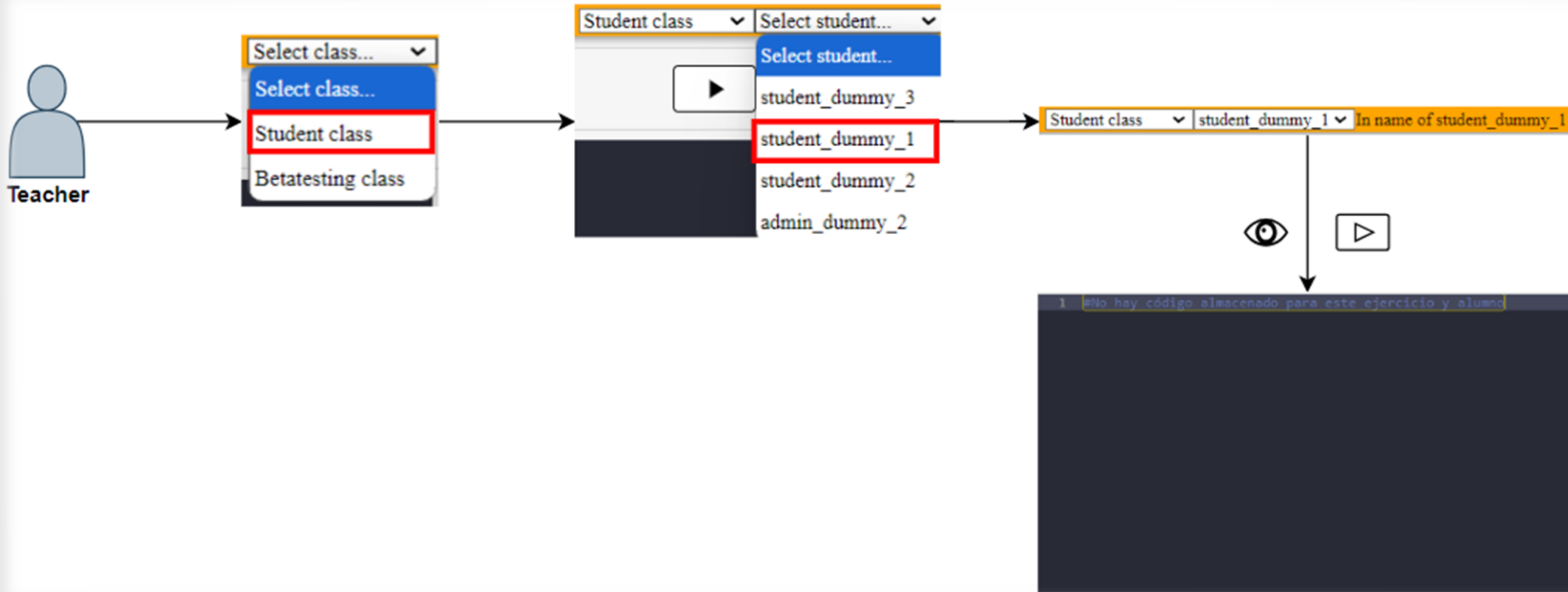
root@44d5059f178f:/#

0%

Press Escape to exit Follow mode

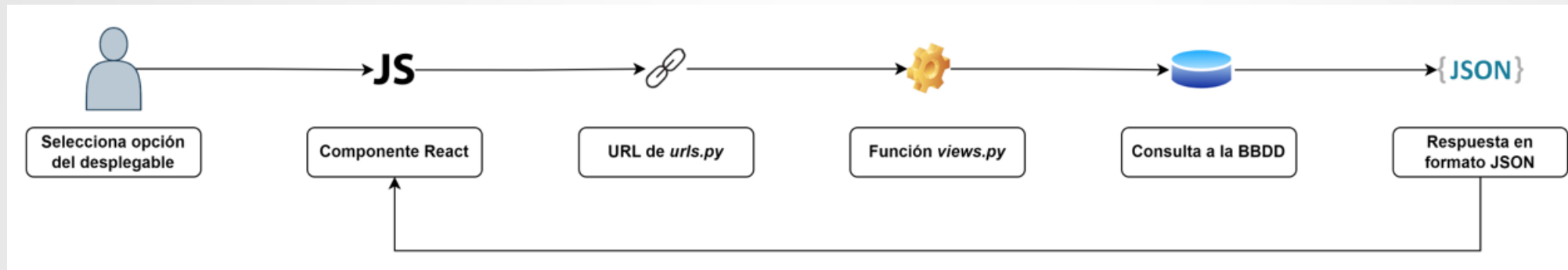
FUNCIONALIDADES PARA PROFESORES

- Flujo que sigue el profesor para visualizar y ejecutar el código de sus alumnos



FUNCIONALIDADES PARA PROFESORES

- Para crear los desplegables y mostrar el código se realizan las siguientes acciones



```
useEffect(() => {
  fetch('/classes/')
    .then(response => {
      if (!response.ok) {
        throw new Error('Network response was not ok ' + response.statusText);
      }
      return response.json();
    })
    .then(data => {
      setclasses(data.classes);
    })
    .catch(error => {
      console.error('Hubo un error:', error);
    });
});
```

```
def get_classes(request):
    user_id = request.user.id
    classes_query = Classes.objects.filter(usuarios=user_id)
    classes = []
    for c in classes_query:
        classes.append(c.nombre_clase)
    return JsonResponse({'classes': classes})
```

```
function showUserCode(code){
  const editorele = ace.edit("code");
  editorele.setValue(code)
}
```

CONCLUSIONES

- Objetivos alcanzados
 - Diferenciación de usuarios gracias a la implementación de Roles
 - Mejora en la gestión de usuarios y contenidos con la aparición de Clases y Cursos
 - Mayor flexibilidad gracias a los nuevos Permisos
 - Nueva interfaz de usuario para integrar las novedades
 - Mayor control de los docentes sobre el trabajo realizado y las horas que dedican sus alumnos
- Trabajos futuros
 - Nuevos gráficos con más datos sobre los alumnos para profesores
 - Mejoras en las páginas de cada curso
 - Soluciones de referencia para los profesores a cada ejercicio

